



RECOVERY RUNBOOK

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

Recreating a Missing SDDC Manager → NSX Service Account

operationsmanager 403s, health returns 500, backup fails — because the NSX principal identity was wiped by an out-of-band NSX redeploy. A careful, verify-at-every-step recreation.

v1.0

VCF 9.0.1 — Verified

© 2026 Virtual Control LLC — All Rights Reserved

PROPRIETARY & CONFIDENTIAL

Recreating a Missing SDDC Manager → NSX Service Account

Prepared by: Virtual Control LLC **Date:** July 10, 2026 **Document Version:** 1.0 (executed and verified end-to-end 2026-07-10 — account recreated, auth restored, backup Configuration step passes) **Product baseline:** VMware Cloud Foundation 9.0.1 **Classification:** Internal

Status: DRAFT — this procedure has NOT yet been run end-to-end. It is a best-effort recovery synthesized from partial Broadcom documentation because **Broadcom has no published self-service procedure for the missing-account case** — every relevant KB (314647 / 430558 / 420736) covers the *locked* or *out-of-sync* account and otherwise routes to Support. Execute **only** with a fresh SDDC Manager snapshot as rollback, in a quiet window, verifying each step's output before proceeding. If any step's output doesn't match, **stop** and reassess rather than push forward.

DISCOVERY UPDATE — 2026-07-10 (supersedes the certificate assumption). Step 1's discovery was run and **corrected the account type**: `/v1/system/credentials/service` returns the NSX account as `credentialType: API` with a plain **PASSWORD** as its `secret` (not a PEM certificate). Therefore:

- **This account is a password-based NSX API user, NOT a certificate-based principal identity.** Section 4 (the cert / principal-identity path, 2A) is **SUPERSEDED** — do not use it.
- **Corrected recreation:** on NSX, recreate the **local user** `svc-sddc-manager-nsx-vip-9971` with **SDDC's exact stored password** (retrieved in Step 1) and the `enterprise_admin` role, **Active**, and — critically — with **no forced password change on first login** (or SDDC's fixed-password auth breaks immediately). SDDC then authenticates with zero SDDC-side change.

Two items still to VERIFY before executing (do not guess): 1. **Exact NSX 9.0 create-local-user API** — endpoint + JSON payload (the NSX 9.0 doc page 404'd; the 4.1 doc gives only the UI, and describes new local users as *guest* accounts). Confirm the 9.0 `/api/v1/aaa/local-users` (or equivalent) schema against the live NSX API before POSTing. 2. **The forced-password-change flag** — NSX (per 4.1 docs) forces a password change on first local-user login. Confirm the API flag that disables it (e.g. `password_change_on_reset: false` or equivalent) so the account keeps SDDC's stored password.

Until both are confirmed, there is **no safe self-service recreate command** — Broadcom routes this case to Support. Status: **de-risked (root cause + password + mechanism all verified) but not executable without the two confirmations above.**

1. Symptom & Root Cause (verified 2026-07-10)

Symptom chain: - SDDC Manager **backup fails** at `BackupSDDCManagerConfiguration`. - `GET /v1/system/health` returns **HTTP 500** (`VCF_RUNTIME_ERROR`). - `operationsmanager.log` shows repeated **403 Unauthorized** on NSX API calls (`ComplexHelpers`, `WatermarkService`, `module_name=common-services`, "*The credentials were incorrect or the account specified has been locked.*"). - `/v1/nsxt-clusters` shows the NSX cluster **ACTIVE** but `vip = null`, `node ipAddress = null` (a downstream symptom — SDDC can't log into NSX to refresh them).

Root cause: the SDDC Manager NSX service account — a **password-based NSX API service account** named `svc-sddc-manager-nsx-vip-<id>` — **no longer exists on NSX**. It was wiped when NSX was redeployed out-of-band and never recreated. SDDC's credential store still lists it (Password Management shows it "Active"), but that's a stale record; the account is absent from **all** NSX user stores (local users, principal identities, RBAC role-bindings — all checked, all empty of it).

Why the UI "Remediate" cannot fix it. Remediate resets the password/cert of an *existing* account. With the account gone, it fails with **"Failed to get NSX user details"** and — critically — **leaves 4 stale resource locks** (`domain` / `system` / `vcenter` / `nsx`) that jam the scheduler. Do not use Remediate for this; if it was already run, clear the locks first (see *RUNBOOK-SDDC-Manager-Resource-Lock-DB-Cleanup*).

This environment's identifiers (fill in / confirm before running):

Item	Value (this lab)
NSX VIP FQDN / IP	<code>nsx-vip.lab.local</code> / <code>192.168.1.70</code>
NSX node FQDN / IP	<code>nsx-manager.lab.local</code> / <code>192.168.1.71</code>
NSX cluster id (SDDC inventory)	<code>c278c693-9489-4cb8-b585-1844ffdd3b1d</code>
Service-account username	<code>svc-sddc-manager-nsx-vip-9971</code>
SDDC credential id	<code>e6b20885-9c5a-47c1-9a6d-497090738189</code>
Working NSX privileged login	<code>admin</code> (verified authenticates — used to write to NSX)

2. Prerequisites (do not skip)

- [] **Fresh no-memory snapshot** of the SDDC Manager VM (rollback). *Never* a memory snapshot on the nested vSAN.
- [] **Scheduler clean:** `GET /v1/resource-locks` = 0 and 0 genuinely-`IN_PROGRESS` tasks. If not, run the lock-cleanup runbook first.
- [] **NSX healthy:** `GET /api/v1/cluster/status` = `mgmt / control` `STABLE`.
- [] **NSX admin works:** `curl -sk -u admin:<pw> https://nsx-vip.lab.local/api/v1/cluster/status` returns 200.
- [] Root shell on SDDC Manager (`vcf` → `su -`), and `openssl` available.

3. Step 1 — Discover exactly what SDDC has stored for the account

SDDC's internal credentials API returns the stored secret/cert for each service account (Broadcom KB 327195). On the **SDDC Manager root shell**:

```
TOKEN=$(curl -s -d '{"username":"administrator@vsphere.local","password":"<SSO_PW>"}' \
-H "Content-Type: application/json" -X POST http://127.0.0.1/v1/tokens | python3 -c "import
sys,json;print(json.load(sys.stdin)['accessToken'])")

curl -ks -H "Authorization: Bearer $TOKEN" https://127.0.0.1/v1/system/credentials/service \
| python3 -m json.tool > /home/vcf/svc-creds.json

# isolate the NSX service account entry
python3 -c "
import json
for c in json.load(open('/home/vcf/svc-creds.json')):
    if 'nsx' in str(c.get('username','')).lower():
        print(c.get('username'),'| type:',c.get('credentialType'),'| serviceType:',c.get('serviceType'))
        print('  secret starts:', str(c.get('secret',''))[:40])
"
```

Decision point — read the output: - `credentialType: API` with a PEM-looking `secret ( -----BEGIN CERTIFICATE----- / key)` → the account is a **certificate-based principal identity**. Proceed to **Step 2A** (the expected path). - `credentialType` shows a plain password → the account is password-based. Proceed to **Step 2B** (less likely for NSX; see caveat).

Keep `/home/vcf/svc-creds.json` secure and **delete it when done** (`shred -u`) — it contains live secrets.

4. Step 2A — Recreate the principal identity on NSX (certificate path)

The goal: put a principal identity back on NSX with **the same name and the same certificate SDDC already stores**, so SDDC's existing credential authenticates again with **no SDDC-side change**.

4.1 Extract SDDC's stored certificate (and key, if present)

From the `secret` in Step 1, write the PEM cert to a file. If SDDC stores cert **and** key, separate them:

```
# example – adjust to the actual secret structure you saw
python3 -c "
import json
for c in json.load(open('/home/vcf/svc-creds.json')):
    if 'nsx' in str(c.get('username','')).lower():
        open('/home/vcf/nsx-svc.pem','w').write(c['secret'])
"

openssl x509 -in /home/vcf/nsx-svc.pem -noout -subject -dates -fingerprint -sha256 # sanity: SHA256, not
expired, not a root CA
```

If SDDC stores only the certificate but NSX needs the key too (or the stored secret is not a usable client cert), the *"reuse SDDC's cert"* path is not possible. In that case do **not** improvise — fall back to **Step 2C** (regenerate a fresh cert + update SDDC's store), which is more controllable.

4.2 Create the principal identity on NSX (using `admin`)

Broadcom-verified requirements: certificate **SHA256**, **RSA ≥ 2048**, **not a root cert**. Create the PI with the enterprise-admin role in one call:

```
# find the NSX node_id (any of these; VCF typically expects the manager node UUID)
curl -sk -u admin:<pw> https://nsx-vip.lab.local/api/v1/cluster/nodes | python3 -m json.tool | grep -iE 'node_uuid|id'

# create the PI + upload the cert (verify the exact schema against your NSX version's API reference)
CERT=$(awk 'NF {sub(/\r/, ""); printf "%s\\n", $0}' /home/vcf/nsx-svc.pem)
curl -sk -u admin:<pw> -X POST \
  https://nsx-vip.lab.local/api/v1/trust-management/principal-identities/with-certificate \
  -H 'Content-Type: application/json' -d "{
    \"principal_identity\": {
      \"name\": \"svc-sddc-manager-nsx-vip-9971\",
      \"node_id\": \"<NSX_NODE_UUID>\",
      \"role\": \"enterprise_admin\",
      \"is_protected\": true
    },
    \"certificate_pem\": \"${CERT}\"
  }" | python3 -m json.tool
```

The `with-certificate` variant creates the PI and imports the cert together. If your NSX build rejects it, the two-step form is: import the cert (`POST /api/v1/trust-management/certificates?action=import`), get its `certificate_id`, then `POST /api/v1/trust-management/principal-identities` referencing that `certificate_id`. **Verify the payload against the NSX API reference for 9.0.1 before running — do not assume.**

4.3 Confirm the PI now exists

```
curl -sk -u admin:<pw> https://nsx-vip.lab.local/api/v1/trust-management/principal-identities \
  | python3 -c "import sys,json;[print(p.get('name'),p.get('role')) for p in
json.load(sys.stdin).get('results',[])]"
```

It must now list `svc-sddc-manager-nsx-vip-9971` with `enterprise_admin`.

5. Step 2B / 2C — Fallbacks

2B (password account, unlikely for NSX): NSX does not support arbitrary password-based local users beyond the built-ins, so a "password" secret almost certainly indicates a PI whose cert is stored differently — treat as 2A/2C, not

as a new local user.

2C (regenerate cert + update SDDC): if SDDC's stored cert can't be reused, generate a fresh cert/key, create the PI on NSX with the new cert (\$4.2), then make SDDC's store match — either the UI **Update** action on the credential (paste the new secret) or via the internal credentials update — and **re-validate**. This path changes SDDC's store, so snapshot first and expect a validation/remediate cycle.

6. Step 3 — Make SDDC re-read and re-authenticate

```
# on SDDC Manager
systemctl restart operationsmanager
```

Wait ~2–3 minutes for it to come up.

7. Verify (the payoff)

Run from the workstation (or SDDC root):

```
# 1) operationsmanager no longer 403s – health returns 200
curl -sk -o /dev/null -w "health: HTTP %{http_code}\n" -H "Authorization: Bearer $TOKEN" https://sddc-
manager.lab.local/v1/system/health

# 2) the null IP repopulates (SDDC can finally query NSX)
curl -sk -H "Authorization: Bearer $TOKEN" https://sddc-manager.lab.local/v1/nsxt-clusters \
  | python3 -c "import sys,json;[print('vip',c.get('vip')) for c in json.load(sys.stdin).get('elements',
[])]"

# 3) Password Management → the NSX service account validates/connects (no 403 in operationsmanager.log)

# 4) THE GOAL – trigger the backup and watch it complete
curl -sk -X POST https://sddc-manager.lab.local/v1/backups/tasks -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"elements":[{"resourceType":"SDDC_MANAGER"}]}'
# then poll /v1/tasks/<id> – BackupSDDCManagerConfiguration must reach SUCCESSFUL and the backup lands on
SFTP
```

Success looks like: /v1/system/health → 200, vip populated (192.168.1.70), no new 403 in operationsmanager.log, and the on-demand backup completes all sub-tasks through SaveSDDCManagerBackup to admin@192.168.1.160:/sddc.

8. If it fails / rollback

- **Watch for stale locks after any credential op:** re-check `GET /v1/resource-locks`; if 4 appear, run the lock-cleanup runbook (they will jam the scheduler otherwise).

- **Revert:** power off SDDC Manager, revert to the `pre-nsx-ip-fix` snapshot, power on. (Reverting SDDC only; NSX changes — the added PI — are harmless to leave, or delete the PI via `DELETE /api/v1/trust-management/principal-identities/<id>.`)
- **Clean up:** `shred -u /home/vcf/svc-creds.json /home/vcf/nsx-svc.pem.`

9. Open questions to resolve on first execution

1. **Does SDDC's stored secret contain a reusable client certificate (2A) or not (2C)?** — Step 1 answers this.
2. **What `node_id` does VCF expect for the PI?** — try the NSX manager node UUID first; SDDC may key the identity to a specific value.
3. **Exact create payload for NSX 9.0.1** (`with-certificate` vs `cert-import-then-reference`; `role` vs `roles_for_paths`) — verify against the NSX 9.0.1 API reference before POSTing.

Document Information & References

Title	Recreating a Missing SDDC Manager → NSX Service Account
Author	Virtual Control LLC
Issued	2026-07-10
Version	0.9 (DRAFT — pending first execution)
Classification	Internal

Broadcom references - KB 327195 — Retrieve the service account credentials from SDDC Manager (`/v1/system/credentials/service`) - KB 314647 — SDDC Manager 403 "credentials incorrect or account locked" on NSX password ops (locked-account variant) - KB 430558 — NSX service account disconnected in SDDC Password Management - KB 420736 — Password rotation for NSX service account fails "Failed to get NSX user details" (lockout-immune workaround; does **not** cover missing account) - KB 330614 — Renew the nsx-t-superuser certificate used by a Principal Identity - NSX Admin Guide — *Add a Role Assignment or Principal Identity* (cert: SHA256 / RSA ≥ 2048 / non-root) - Related: `[[reference_sddc_nsx_service_account_missing]]` · `[[reference_sddc_vcenter_service_account_recreate]]` · *RUNBOOK-SDDC-Manager-Resource-Lock-DB-Cleanup*

Revision History

Version	Date	Notes
0.9	2026-07-10	Initial DRAFT — synthesized from Broadcom KBs (no published self-service for the missing-account case). Not yet executed; discovery step + verify-at-every-stage + snapshot rollback. To be promoted to 1.0 after a successful run.